Listing 10.26 Configuring the output binding to be transactional

```
spring:
  cloud:
    function:
      definition: dispatchOrder
    stream:
      bindings:
        dispatchOrder-in-0:
          destination: order-dispatched
          group: ${spring.application.name}
        acceptOrder-out-0:
          destination: order-accepted
      rabbit:                                  ❶
        bindings:
          acceptOrder-out-0:
            producer:
              transacted: true                 ❷
```

❶ RabbitMQ-specific configuration for the Spring Cloud Stream bindings

❷ Makes the acceptOrder-out-0 binding transactional

Now you can write new integration tests for the supplier and the consumer, like we did for the functions in Dispatcher Service. I'll leave the autotests to you, since you have the necessary tools now. If you need inspiration, check out the source code accompanying this book (Chapter10/10-end/order-service).

You will also need to import the configuration for the test binder (`@Import(TestChannelBinderConfiguration.class)`) in the existing `OrderServiceApplicationTests` class to make it work.

We've had a nice journey through event-driven models, functions, and messaging systems. Before wrapping up, let's look at the order flow in action. First, start RabbitMQ, PostgreSQL (`docker-compose up -d polar-rabbitmq polar-postgres`), and Dispatcher Service (`./gradlew bootRun`). Then run Catalog Service and Order Service